

Git and GitHub

Versioning

"FINAL".doc



↪ FINAL.doc!



↪ FINAL_rev.2.doc

Versioning



FINAL_rev.6.COMMENTS.doc



FINAL_rev.8.comments5.
CORRECTIONS.doc

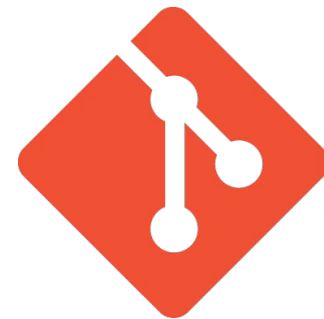
Versioning

JORGE CHAM © 2012



FINAL_rev.18.comments7.
corrections9.MORE.30.doc

FINAL_rev.22.comments49.
corrections.10. #@\$%WHYDID
ICOMETOGRADSCHOOL?????.doc



Git

- System to keep track of changes that happen across the set of files
- A.K.A. version control tool



Github

- Online service to run Git in the cloud
- You version your files with Git and then share and store using github
- Versioned files are easily shared across multiple computers

Git vs Google Docs

	<i>saves current state ...</i>	<i>keeps track of ...</i>
Git	on request	directory tree (multiple files and their position)
Google Docs	automatically	single file

The screenshot displays the Google Docs interface. On the left, a document titled 'Today, 1:45 PM' is open, showing a text editor with a green highlight and the text 'Here I have added some text.' Below this, a section titled 'What Is Google Docs' is visible, containing a paragraph about Google Docs being an online word processor. On the right, the 'Version history' sidebar is open, showing a list of versions. The top version is 'March 9, 1:45 PM', labeled 'Current version' and attributed to 'Tina Sieber'. Below it is 'First Draft' from 'March 9, 1:43 PM', also by 'Tina Sieber'. The bottom version is 'March 9, 12:57 PM' by 'Tina Sieber'. A toggle switch for 'Only show named versions' is visible at the top of the sidebar.

← Today, 1:45 PM

100%

Version history

Only show named versions

Today

- ▶ **March 9, 1:45 PM**
Current version
■ Tina Sieber
- First Draft
March 9, 1:43 PM
■ Tina Sieber
- March 9, 12:57 PM
■ Tina Sieber

-Here I have added some text.

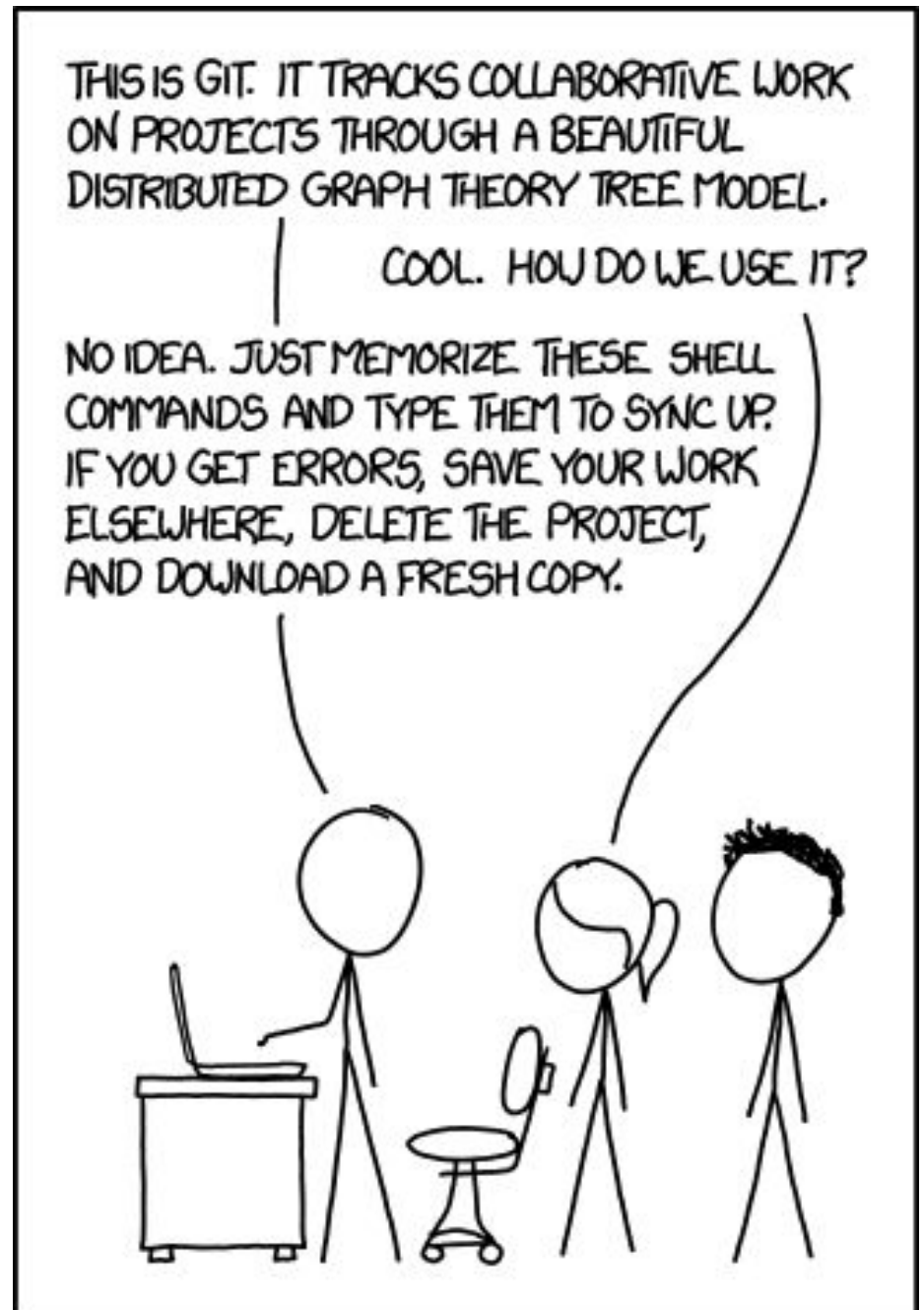
What Is Google Docs

Google Docs is Google's online word processor. What's its competitor, Microsoft Word, are its collaborative features. documents across platforms and work on them together in. Your collaborators don't even need a Google account to view or share with them.

Git and Github: upsides

- collaboration
 - sharing
 - peer review
- backup
 - can go back to any version
 - simple and secure way to distribute to other machines

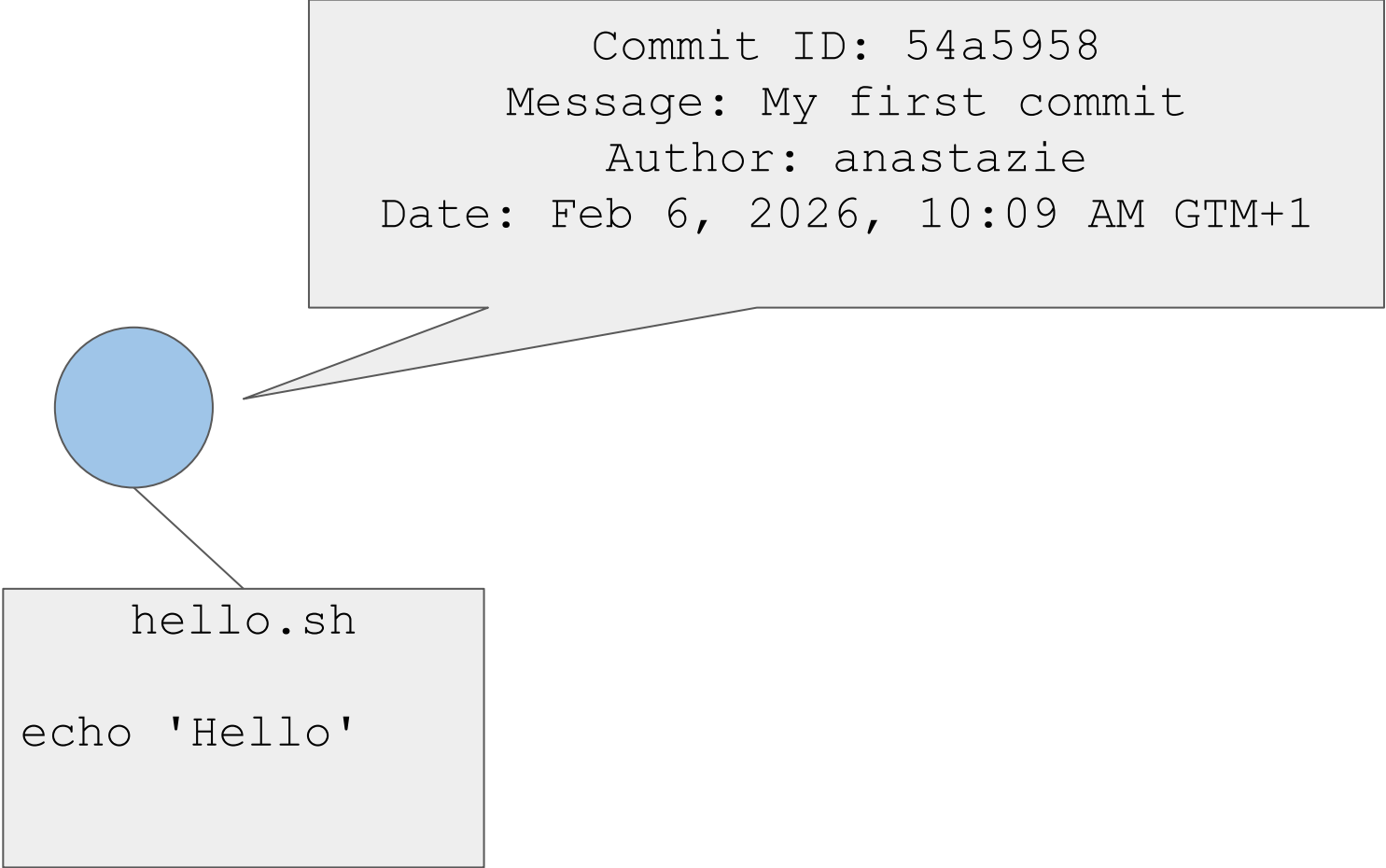
Git: downsides



Vocabulary

- Repository
 - Project's folder
 - A repository contains all of the project files (including documentation), and stores each file's revision history
 - Repositories can have multiple collaborators and can be either public or private
- Commit
 - Records changes (snapshots) to one or more files
 - Core building block units of a Git project timeline
- Branch
 - A branch is a parallel version of a repository. It is contained within the repository, but does not affect the primary or main branch allowing you to work freely without disrupting the "live" version.
- More terms [here](#)

Commits



Commit ID: 54a5958
Message: My first commit
Author: anastazie
Date: Feb 6, 2026, 10:09 AM GMT+1

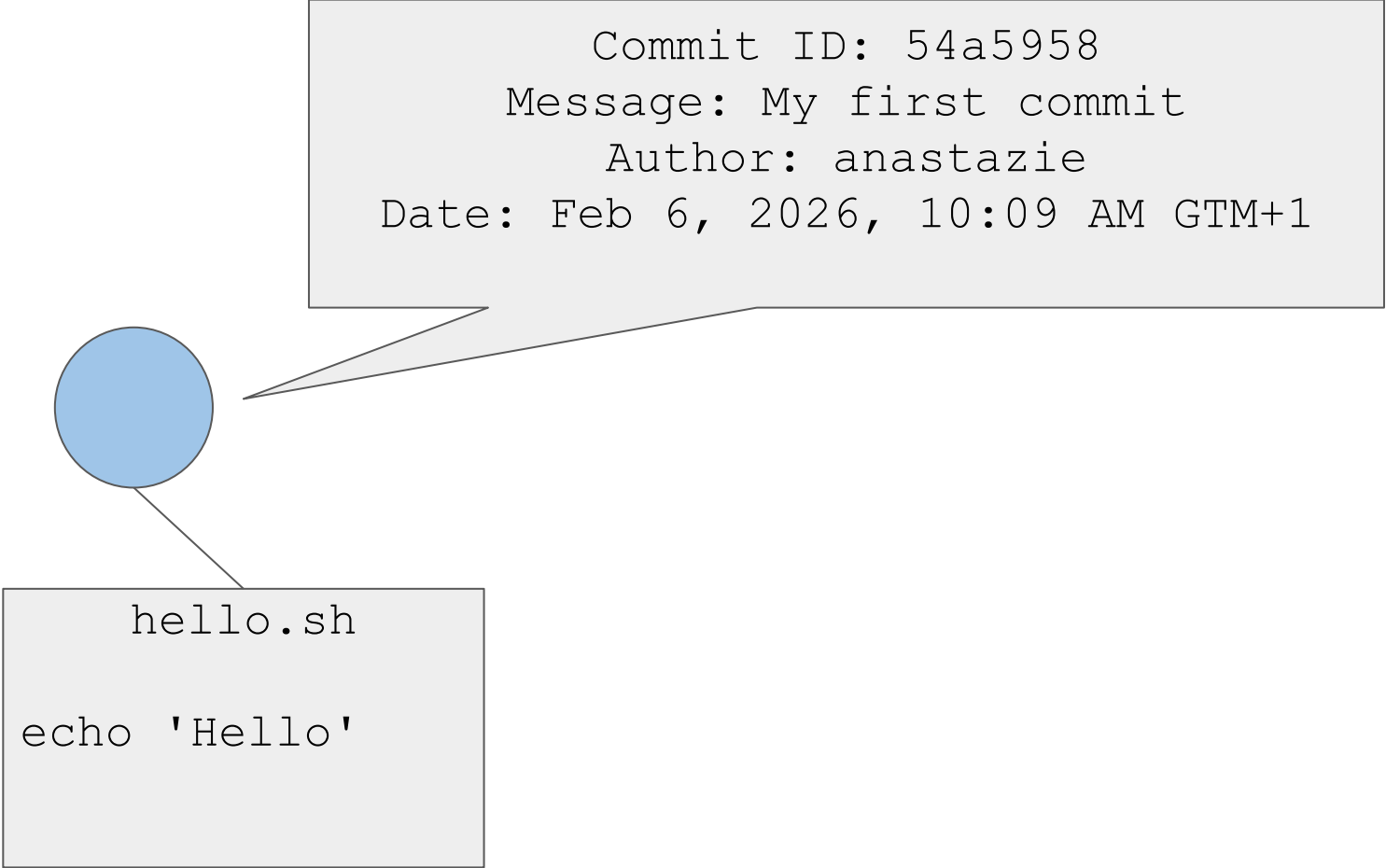
A diagram illustrating a commit. A light gray rectangular box at the top right contains commit metadata. A blue circle is positioned to the left of this box, with a line connecting it to a second light gray rectangular box at the bottom left. This second box contains the filename 'hello.sh' and its contents 'echo 'Hello''.

hello.sh

echo 'Hello'

Commits

Commit identification

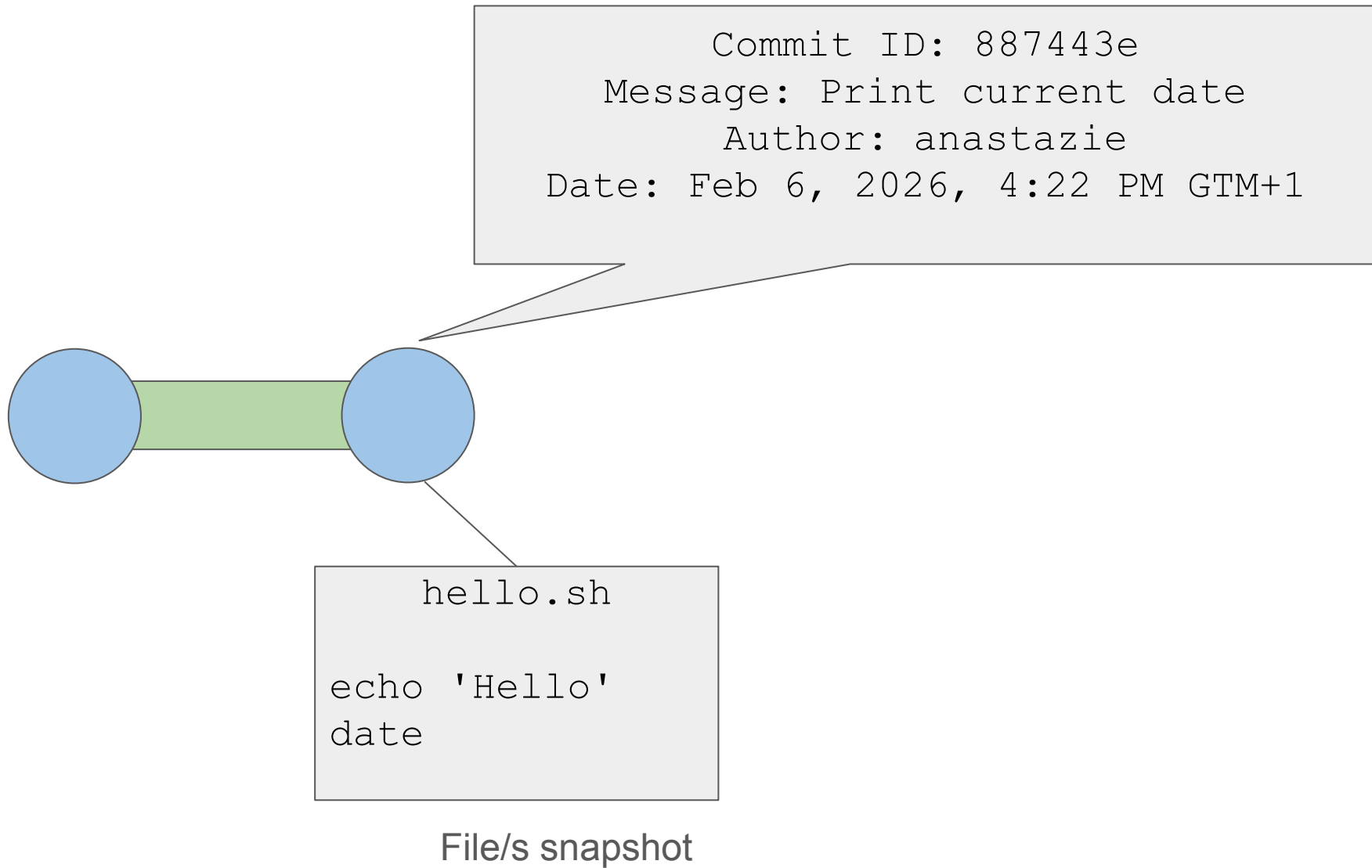


Commit ID: 54a5958
Message: My first commit
Author: anastazie
Date: Feb 6, 2026, 10:09 AM GMT+1

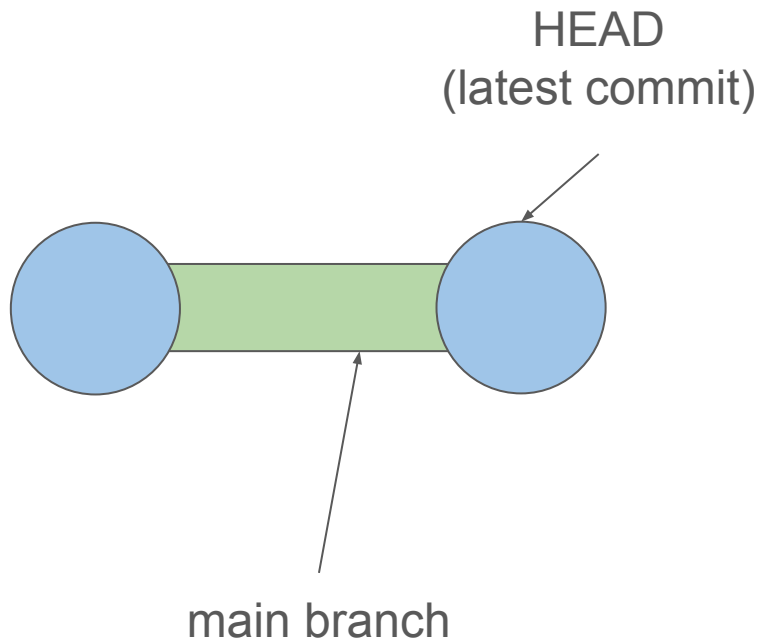
hello.sh
echo 'Hello'

File/s snapshot

Commits

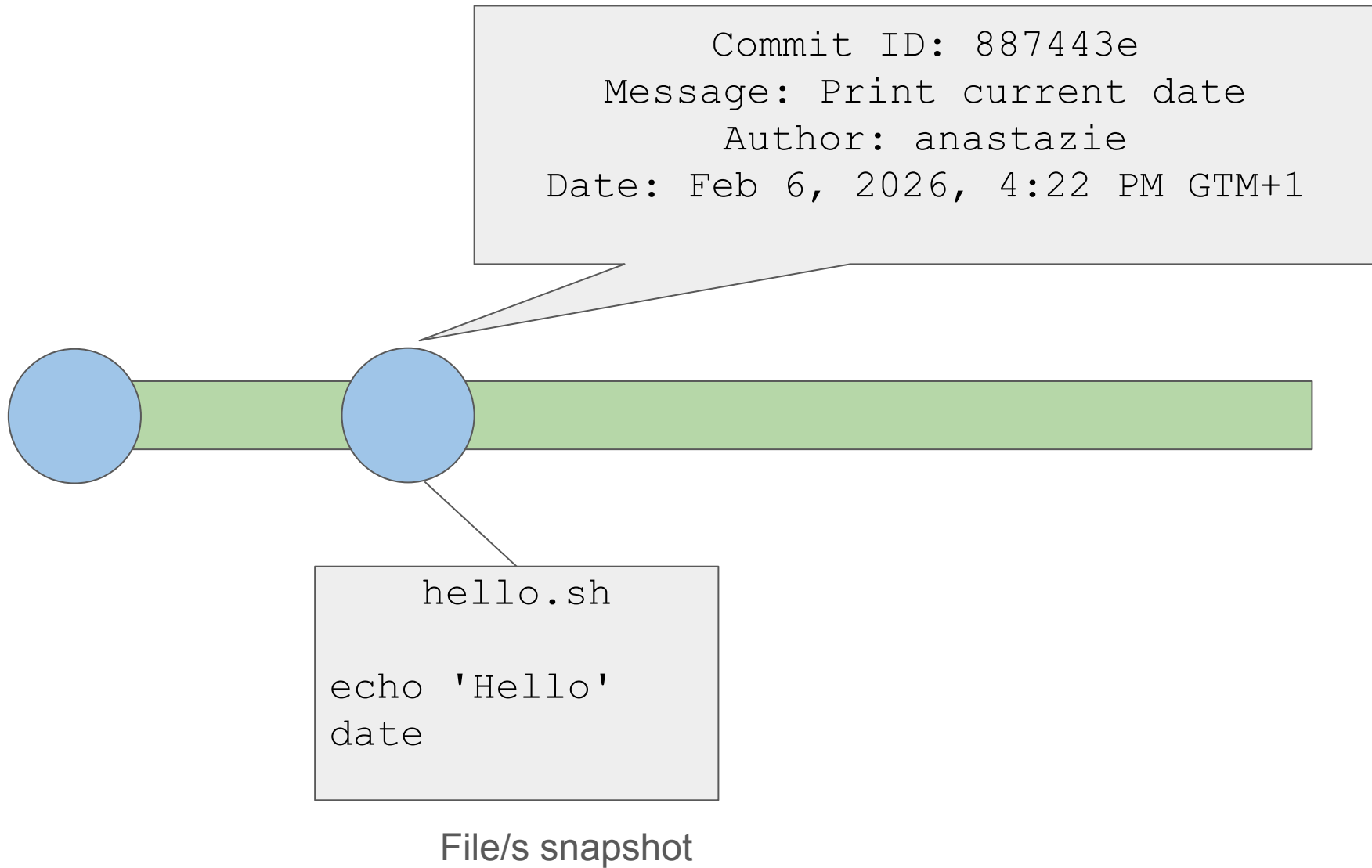


Commits



File/s snapshot

Commits



Local workflow

- initialize a repository
 - `git init`

Working copy	Staging area	Commits
my-file.txt data .gitignore		

Local workflow

- initialize a repository
 - `git init`
- add files to staging area (where you prepare the next commit)
 - `git add my-file.txt`

Working copy	Staging area	Commits
my-file.txt data .gitignore	my-file.txt	



Local workflow

- initialize a repository
 - `git init`
- add files to staging area (where you prepare the next commit)
 - `git add my-file.txt`
- create a commit
 - `git commit -m 'change this and that'`

Working copy	Staging area	Commits
my-file.txt data .gitignore		my-file.txt 3c4a.. change this...

Diff

A shorthand for “differences”. Usually between working copy and the last commit.

‘-’ means line removed

‘+’ means line added

A sample from course materials repo:

```
``pull`` will do a merge for you
```

```
-  ``git checkout --theirs``
```

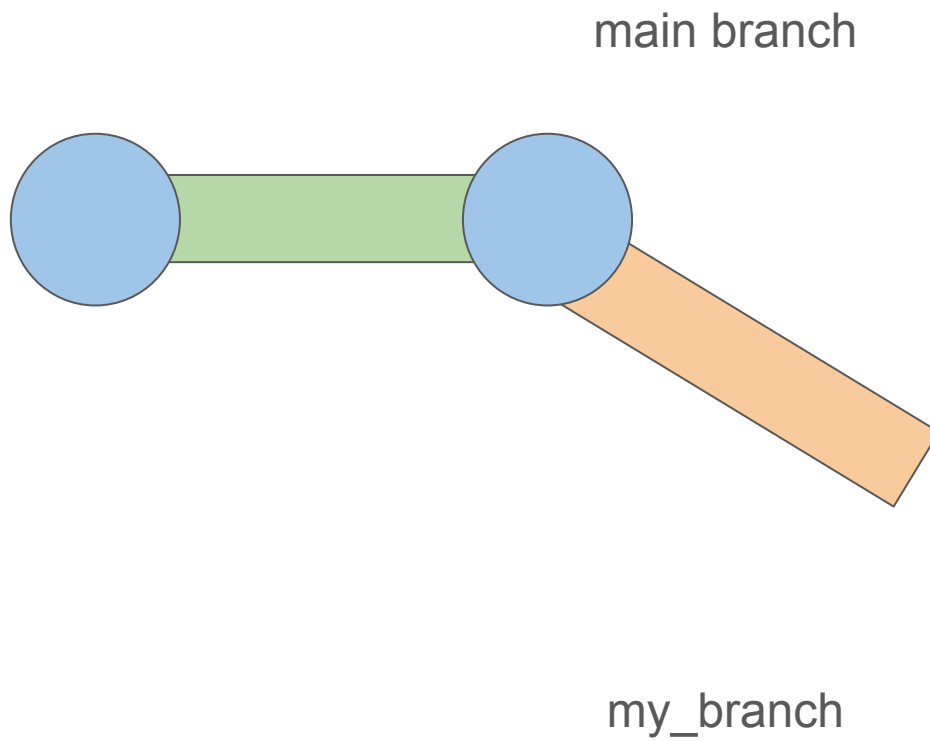
```
+  ``git checkout --theirs {file path}``
```

```
    in conflict, choose the version from the remote branch
```

Branches

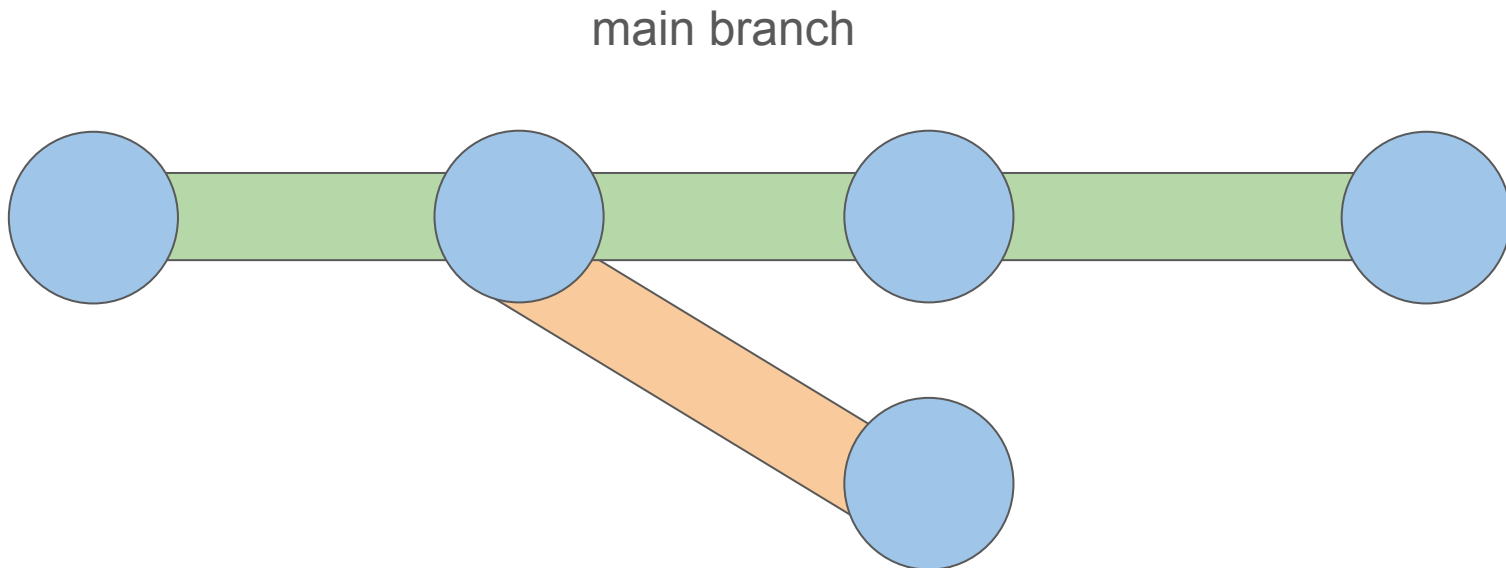
- branches help you manage multiple versions of your code
- important for programmers, but we can't ignore it completely
- we'll be using one local branch called **main**
- on a remote, there is a twin branch, **origin/main**

Commits



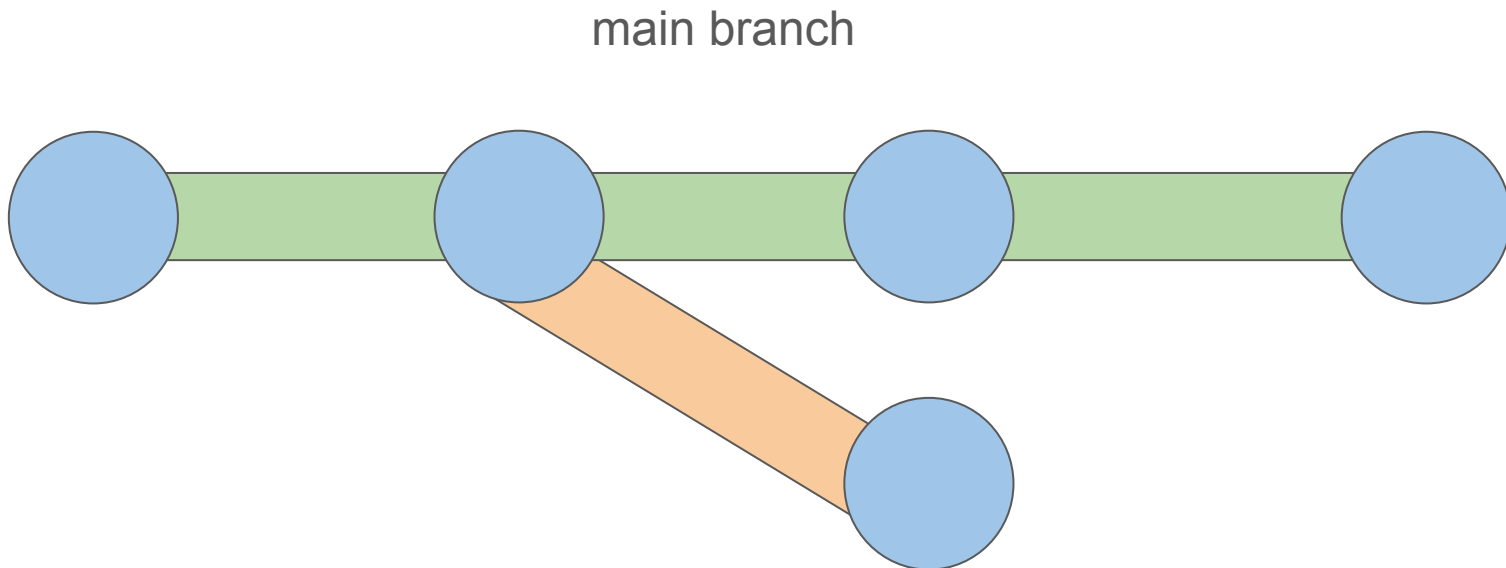
Commits

Working on your branch
does not affect the main
branch



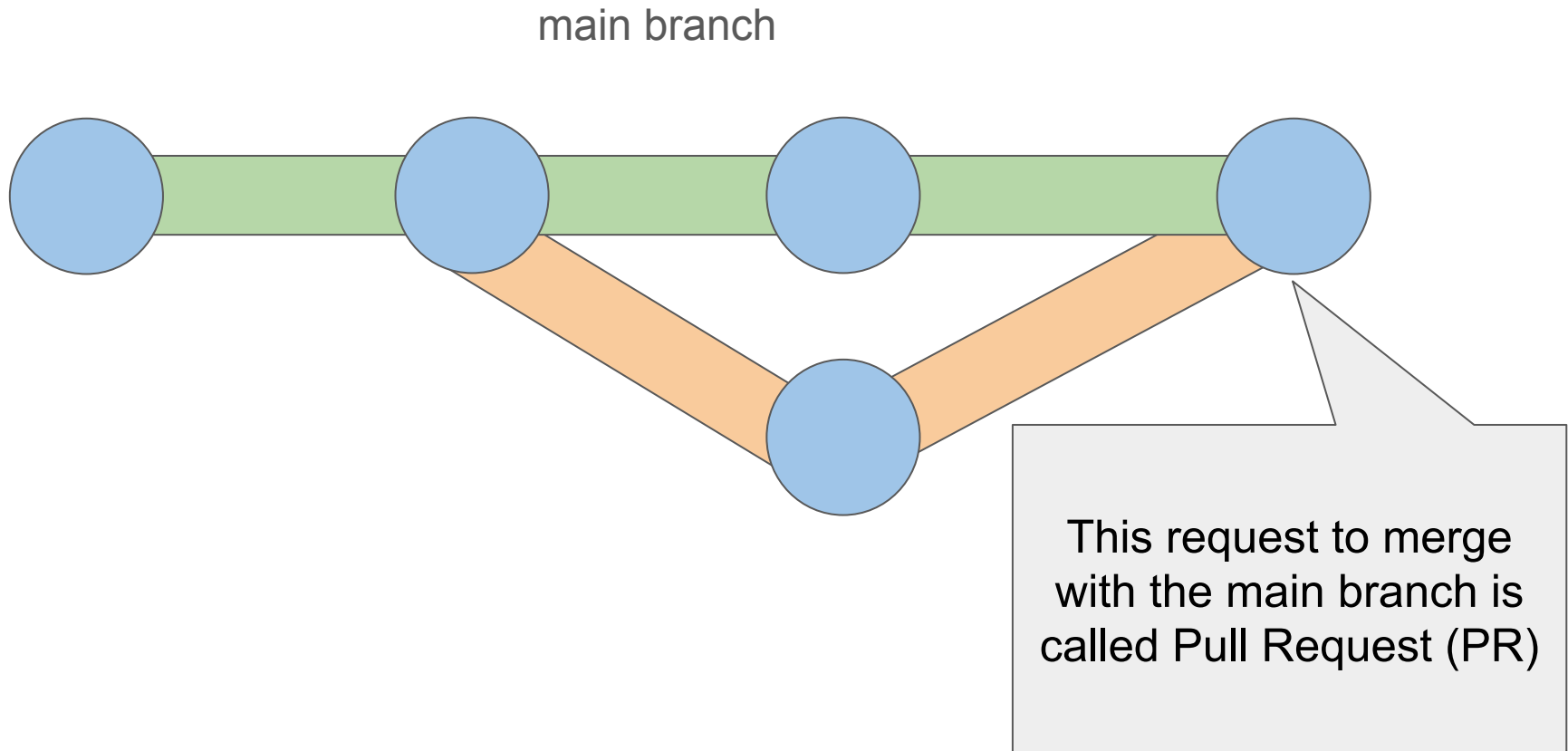
Commits

Working on your branch
does not affect the main
branch



Commits

Once you are done working on your branch you can merge it to the main branch



GitHub

“Social part of programming”



Pushing to a remote [server]

- add a remote
 - `git remote add origin {where}`
- first push - also create a remote branch from the local one
 - `git push -u origin local-branch`
- push - upload commits
 - `git push`



Pulling from a remote

- download commits and merge into working copy
 - `git pull`

Pulling from a remote

- download commits and merge into working copy
 - `git pull`
- simple as that, unless there is a **merge conflict**



Merge conflict

- Changes were made in the same place in the same file in your branch and main branch
- Git cannot decide on its own which version to keep
- You need to decide what to do
 - Use main branch version
 - Use your branch version
 - Combine both

Merge conflict



hello.sh on my_branch

```
1 echo 'Hello'
2 date
3 ls -l ~ | wc -l
```

hello.sh on main

```
1 echo 'Hello'
2 cd ~
3 mkdir data
```

Resolving conflicts - binary files

- check what's conflicted
 - `git status`
- binary files are simple, choose either the incoming (theirs) or ours
 - `git checkout --ours binary-file.jpg`
 - `git add binary-file.jpg`
- commit the resolution, without any message
 - `git commit`
 - recently available: `git merge --continue`

Resolving conflicts - text files

- check what's conflicted
 - `git status`
- text files need to be edited
 - `nano my-file.txt`
 - `git add my-file.txt`
- commit the resolution, without any message
 - `git commit`
 - recently available: `git merge --continue`

Resolving conflicts

In text files, if both branches modify the same line, a conflict has to be resolved by deleting all the unwanted text (also the marks).

Here are lines that are either unchanged from the common ancestor,
or cleanly resolved because only one side changed,
or cleanly resolved because both sides changed the same way.

```
<<<<<< yours:sample.txt
```

```
Conflict resolution is hard;  
let's go shopping.
```

```
=====
```

```
Git makes conflict resolution easy.
```

```
>>>>>> theirs:sample.txt
```

```
And here is another line that is cleanly resolved or unmodified.
```

What have we learned?

- We are using Git to have explicit versioning of directory tree
- Git makes snapshot of one or multiple files and saves them as a commit
- Repository is a project inside which git tracks changes
- Branches allows multiple collaborators to work in the same repository without disrupting the main branch
- You can use Github to share and store your repository. To do that you push your local branch to remote